

System Testing

Unit, integration, performance and edge case testing

System Testing Documentation

Project: FitBuddy — AI-Powered Personal Fitness Coach

Testing Overview

FitBuddy was tested across three layers: **Unit Testing**, **Integration Testing**, and **Performance Testing**. All tests validate the three core AI scenarios and the supporting infrastructure.

1. Unit Testing

Test Cases: Backend API Routes

Test ID	Test Case	Expected Result	Status
UT-01	Health check endpoint returns {"status": "ok"}	HTTP 200, status = ok	■ PASS
UT-02	POST /api/generate-plan with valid data	HTTP 200, plan JSON returned	■ PASS
UT-03	POST /api/generate-plan with missing name	HTTP 422 (validation error)	■ PASS

UT-04	POST <code>/api/generate-plan</code> with invalid weight type	HTTP 422	■ PASS
UT-05	POST <code>/api/update-plan</code> with valid user_id + feedback	HTTP 200, updated plan	■ PASS
UT-06	POST <code>/api/update-plan</code> with non-existent user_id	HTTP 404	■ PASS
UT-07	POST <code>/api/nutrition-tip</code> with valid goal	HTTP 200, nutrition JSON	■ PASS
UT-08	GET <code>/api/user/{id}/history</code> with valid user_id	HTTP 200, user + plans + tips	■ PASS
UT-09	GET <code>/api/user/{id}/history</code> with non-existent user	HTTP 404	■ PASS

Test Cases: Database Operations

Test ID	Test Case	Expected Result	Status
DB-01	Database tables created on app start	Tables exist in SQLite	■ PASS
DB-02	User record inserted on plan generation	User ID returned and stored	■ PASS
DB-03	Plan JSON stored correctly	plan_json is valid parseable JSON	■ PASS
DB-04	Feedback stored with updated plan	feedback field populated	■ PASS
DB-05	Foreign key user_id in workout_plans	References valid user	■ PASS

Test Cases: AI Response Parsing

Test ID	Test Case	Expected Result	Status
AI-01	Gemini returns plain JSON	Parsed correctly	■ PASS
AI-02	Gemini returns JSON in markdown fences	Fence stripped, parsed correctly	■ PASS
AI-03	Gemini returns JSON with <code>json</code> tag	Tag stripped, parsed correctly	■ PASS
AI-04	Invalid JSON from Gemini	HTTP 500 with "AI response parsing error"	■ PASS

2. Integration Testing

End-to-End Flow: Scenario 1 — Generate Plan

Test: Full Plan Generation Flow Step 1: User opens app → Checks initial state (goal=general_wellness, intensity=medium default) Step 2: User fills Name, Age, Weight Step 3: User selects "Weight Loss" goal Step 4: User selects "High" intensity Step 5: Clicks "Generate My 7-Day Plan" Step 6: Loading state appears (spinner + progress bar) Step 7: Plan renders with 7 day cards, User ID banner, weekly tips Step 8: User ID is auto-filled in Feedback and Nutrition tabs Result: ■ PASS — Full flow works correctly

End-to-End Flow: Scenario 2 — Update Plan

Test: Plan Update with Feedback Flow Step 1: User navigates to "Update Plan" tab Step 2: User ID is pre-filled from Session 1 Step 3: User clicks feedback chip "■ More cardio" Step 4: Textarea auto-fills with chip text Step 5: Clicks "Update My Plan" Step 6: Loading state appears Step 7: Updated plan renders with "Plan Updated!" banner Step 8: Feedback text is displayed in banner Result: ■ PASS

End-to-End Flow: Scenario 3 — Nutrition Tips

Test: Nutrition Tip Generation Flow Step 1: User navigates to "Nutrition Tips" tab Step 2: Selects goal from dropdown (e.g., "Muscle Gain") Step 3: Clicks "Get My Nutrition Tip" Step 4: Loading state appears Step 5: Nutrition card renders with title, tip, food lists, hydration, recovery Result: ■ PASS

3. UI/UX Testing

Test	Expected Behavior	Status
Tab switching	Only active panel visible	■ PASS
Goal card selection	Previous selection deselected, new highlighted	■ PASS
Intensity button selection	Color correctly changes per level	■ PASS
Day card click expand	Card expands to show full details	■ PASS
Toast success notification	Green toast appears for 4 seconds	■ PASS
Toast error notification	Red toast appears with error message	■ PASS
Animated counters	Hero stats count from 0 to target	■ PASS
Spinner button state	Button disabled + spinner shown during fetch	■ PASS
Responsive at 375px width	All elements stack correctly on mobile	■ PASS
Form validation	Empty fields trigger warning toast	■ PASS

4. Performance Testing

Test: API Response Times

Endpoint	Test Runs	Min	Max	Avg
GET /api/health	10	12ms	45ms	22ms
POST /api/generate-plan	5	9.2s	18.4s	13.1s

POST /api/update-plan	5	8.8s	16.2s	12.0s
POST /api/nutrition-tip	5	4.5s	9.8s	6.3s

Test: Concurrent Users (Basic)

Concurrent Users	Scenario	Result
1	Plan generation	■ Success
3	Mixed (generate + nutrition)	■ Success
5	All scenario 1	■ Success (slightly slower)

Note: SQLite handles low concurrency. For production scale, PostgreSQL is recommended.

5. Edge Case Testing

Edge Case	Expected Behavior	Status
Very young user (age=13)	Plan generated with appropriate exercises	■ PASS
Very heavy user (weight=150kg)	Plan generated with appropriate difficulty	■ PASS
Single-word feedback	Plan updated based on minimal feedback	■ PASS
Empty feedback textarea	Warning toast, no API call	■ PASS
Invalid user ID (9999)	404 error with toast notification	■ PASS
No goal selected in nutrition	Warning toast shown	■ PASS
AI returns partial JSON	JSON parse error caught, 500 returned	■ PASS

6. Security Testing

Test	Expectation	Result
API key in source code	Should be env var	■ Env var used
SQL injection in user inputs	Parameterized queries prevent injection	■ Safe
XSS in plan content	Content rendered as text, not raw HTML	■ Safe
CORS configured	All origins allowed (dev mode)	■■ Note: Restrict in production

7. Known Limitations

- AI Latency:** Plan generation takes 10-20s due to Gemini API response time. Progress bar provides UX feedback.
- No Auth:** No user authentication — User IDs are guessable. Fine for demo, not for production.
- SQLite Concurrency:** Not suitable for high-traffic production use. Recommend PostgreSQL.
- API Key Exposure:** Default key fallback should be removed in production deployments.